

Authored by : Arpit Raj , Lnmiit jaipur

Ch 7 Data Abstraction and 3 schema architecture

Data Abstraction

-> process of hiding unnecessary details of data storage (like which block stores record, which page contains it, which index, which file offset occupies it, B+ tree or heap file)

It exposes only the info required by different categories of user

ANSI - SPARC 3 schema architecture

different users needs different views

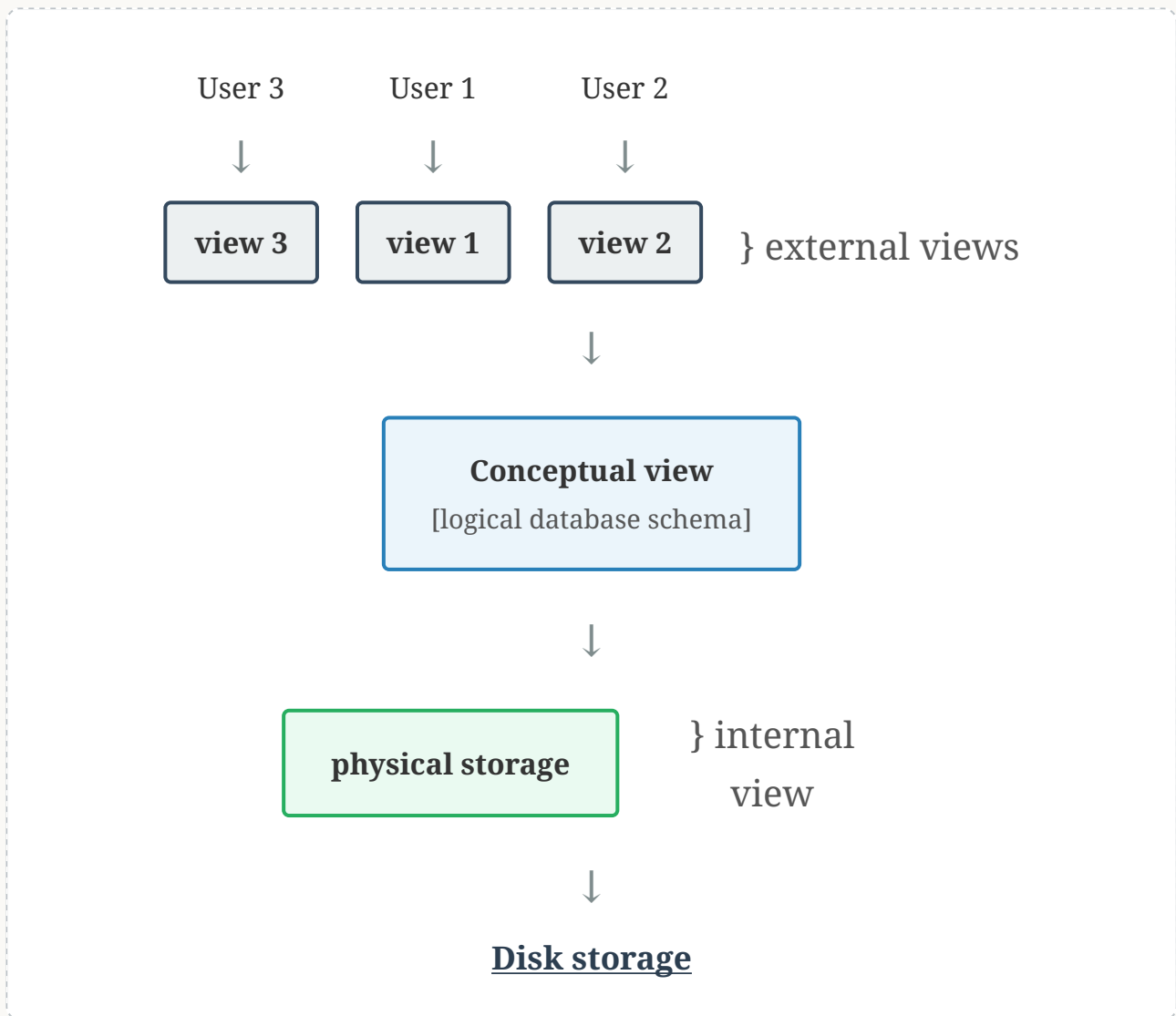
Eg

Aadz may need just her CGPA, courses

professor may need attendance, marks

administrator may need indexes, storage, users, backups

developer may need pages, blocks, disk layout



external view → multiple user specific view exposes only the required subset of database

conceptual view → Tables, columns, relation, constraints, keys, data types complete logical structure of database

Internal view → how the database is physically stored on secondary storage
Pages, blocks, B trees, hash indexes, buffer pools

process →

1. read external view
2. map to conceptual schema
3. map to physical storage
4. retrieve data

5. convert back to rows

Data Independence

-> ability to change or modify one level of database without changing any higher level

Physical data independence

-> ability to change internal schema without changing conceptual schema

Eg Heap file to B+ Tree Index

SQL didnt change, no application change

so one can change storage structure, Indexing strategy, file organization without affecting database schema

Logical Data independence

modify conceptual schema without affecting external view

harder to achieve as logical changes or applications depend on schemas

Eg

ID	Name	CG
006	Aadz togi	7.9
011	Arpit Raj	7.7

```
SELECT NAME  
FROM STUDENTS  
where ID = 006
```

now suppose schema is changed

ID	Fname	Lname	cg
006	Aadz	togi	7.9
011	Arpit	Raj	7.7

```
SELECT NAME  
FROM STUDENTS
```

}

ERROR

application is dependent on logical schema

now every application has to be modified which is very expensive

with logical data independence we can :-

```
CREATE VIEW student view AS  
SELECT  
    studentID,  
    CONCAT(FirstName, ' ', LastName) AS NAME,  
    CGPA  
FROM Students
```

Importance of data abstraction

without abstraction every storage change would need rewriting application. hence with abstraction we get

- maintainability
 - portability
 - scalability
-

Practice Questions

Q1

What is data abstraction? Why is it necessary?

A1

Data abstraction is the process of hiding low-level implementation details while exposing only the information required by users. It simplifies application development, improves security, enables storage optimizations, and provides data independence.

Q2

Explain the ANSI-SPARC Three-Schema Architecture.

A2

The ANSI-SPARC Three-Schema Architecture divides the database into three abstraction levels:

- External Level: Multiple user-specific views of the database.
- Conceptual Level: The complete logical schema describing entities, relationships, and constraints.
- Internal Level: The physical storage representation, including pages, indexes, and file organization.

Mappings between these levels allow changes at one level with minimal impact on others.

Q3

Differentiate between the external, conceptual, and internal levels with examples.

A3

External Level: HR sees employee salaries; employees see only their own profiles.

Conceptual Level: Defines tables such as Employees(ID, Name, Salary) and their relationships.

Internal Level: Specifies how those tables are stored using pages, indexes, and record layouts on disk.

Q4

What is data independence?

Why is it important?

A4

Data independence is the ability to modify one schema level without affecting higher levels. It reduces maintenance costs, allows performance optimizations, and isolates application code from storage or schema changes.

Q5

Differentiate between logical and physical data independence.

Give two examples of each.

A5

Physical Data Independence

- Change from heap files to B+ Tree storage.
- Add or remove indexes.
- Move data to faster storage devices.

Logical Data Independence

- Split a Name column into FirstName and LastName.
- Introduce a new table while preserving existing views.

Physical data independence is generally easier to achieve than logical data independence.

Q6

Suppose a DBA creates a new B+ Tree index on the StudentID column.

Which level changes?

Will applications need to be modified?

Why?

A6

Creating a B+ Tree index changes the internal schema. This is an example of physical data independence. Applications do not need modification because the logical schema and SQL interface remain unchanged.

Q7

Suppose a table is redesigned by splitting the Name column into FirstName and LastName.

Which type of data independence is involved?

Is it easier or harder to preserve application compatibility?

A7

Splitting Name into FirstName and LastName is a conceptual schema change, so it relates to logical data independence. Preserving compatibility is harder because application queries may reference the original column unless compatibility views or other mechanisms are provided.